

🔑 GIT ESENCIAL

<code>git clone <repo></code>	Clonar repositorio
<code>git add . && git commit -m ""</code>	Preparar y confirmar cambios
<code>git push / git pull</code>	Subir / bajar cambios
<code>git branch / checkout -b <r></code>	Listar / crear rama
<code>git merge <rama></code>	Fusionar rama en la actual
<code>git log --oneline</code>	Historial compacto
<code>git tag v1.0 && git push --tags</code>	Etiquetar versión

🐳 DOCKER ESENCIAL

<code>docker build -t app:1.0 .</code>	Construir imagen
<code>docker images / docker ps</code>	Listar imágenes / contenedores
<code>docker logs -f <cont></code>	Seguir logs
<code>docker exec -it <cont> sh</code>	Entrar al contenedor
<code>docker stop / rm <cont></code>	Detener / eliminar
<code>docker compose up -d / down</code>	Levantar / tumbar stack
<code>docker system prune -f</code>	Limpiar recursos huérfanos

🔨 TERRAFORM ESENCIAL

<code>terraform init</code>	Inicializar (descargar providers)
<code>terraform validate / fmt</code>	Validar sintaxis / formatear
<code>terraform plan</code>	Previsualizar cambios
<code>terraform apply</code>	Aplicar infraestructura
<code>terraform destroy</code>	Eliminar todos los recursos
<code>terraform state list</code>	Listar recursos del estado

🏠 JENKINS · COMANDOS

<code>java -jar jenkins.war</code>	Iniciar Jenkins standalone
<code>jenkins-plugin-cli --list</code>	Plugins instalados
<code>jenkins-plugin-cli --plugins git docker-workflow</code>	Instalar plugins vía CLI
<code>curl -X POST JENKINS_URL/job/<job>/build</code>	Disparar build (API REST)
<code>curl JENKINS_URL/job/<job>/lastBuild/api/json</code>	Consultar estado del último build
<code>curl -X POST JENKINS_URL/job/<job>/doDelete</code>	Eliminar un job
<code>curl JENKINS_URL/crumbIssuer/api/json</code>	Obtener token CSRF
<code>docker restart jenkins</code>	Reiniciar contenedor Jenkins

⚠️ **CSRF:** Jenkins exige un *crumb* para peticiones POST. Se obtiene con `curl .../crumbIssuer/api/json` y se envía en el header `Jenkins-Crumb: <token>`.

🌱 CONCEPTOS JENKINS

Pipeline Declarativo: estructura rígida con `pipeline`, `stages`, `steps`. Más legible y mantenible.
Pipeline Scripted: Groovy puro, más flexible pero más complejo.
Job / Build: job = tarea configurada; build = ejecución concreta de un job.
Trigger: polling SCM (H/2 * * * * *), webhooks, o disparo manual.
Agent: dónde se ejecuta el pipeline (`agent any`, `agent docker`, `agent none`).

🔧 PLUGINS VITALES

Git: clonar repositorios.
Docker Workflow: construir imágenes.
Credentials Binding: inyectar secretos.
Prometheus: exponer métricas.
Blue Ocean: UI moderna de pipelines.

🏠 TIPOS DE JOBS

Freestyle: interfaz web, scripts sueltos.
Pipeline: Jenkinsfile (recomendado).
Multibranch: un pipeline por rama.

📄 JENKINSFILE DECLARATIVO

```
pipeline {
  agent any
  environment { APP_NAME = "miapp" }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build') { steps { sh 'npm ci' } }
    stage('Test') { steps { sh 'npm test' } }
    stage('Package') { steps { sh "docker build -t ${APP_NAME}:${BUILD_ID} ." } }
    stage('Deploy Staging') {
      steps { sh "docker run -d --name ${APP_NAME}-staging ${APP_NAME}:${BUILD_ID}" }
    }
    stage('Smoke Test') {
      steps {
        sleep 10
        sh "curl -s http://staging:3000/health | grep healthy"
      }
    }
    stage('Approval') {
      steps { input message: '¿Desplegar a producción?', ok: 'Desplegar' }
    }
    stage('Deploy Production') {
      steps { sh "docker run -d --name ${APP_NAME}-prod ${APP_NAME}:${BUILD_ID}" }
    }
  }
  post {
    success { echo '✅ Pipeline completado' }
    failure { echo '❌ Pipeline fallido' }
  }
}
```

💡 **Notas:** BUILD_ID = número único del build (mejor que latest). `sleep` da tiempo a arrancar antes el smoke test. `input` detiene el pipeline hasta que un humano pulsa "Desplegar" (Continuous Delivery).

📊 MÉTRICAS CLAVE (PROMQL)

Métrica	Expresión
CPU host	<code>100 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m]))*100</code>
RAM usada %	<code>(1 - node_memory_MemAvailable/...Total)*100</code>
Disco /	<code>(1 - node_filesystem_avail/...size)*100</code>
Contenedores up	<code>count(container_last_seen)</code>
Builds éxito	<code>default_jenkins_builds_success_total</code>
Builds fallo	<code>default_jenkins_builds_failed_total</code>

💡 Usar `rate(... [5m])` para suavizar picos.
`container_memory_working_set_bytes` (memoria real, excluye cachés).

⚡ PIPELINE CI/CD TÍPICO



📖 CONCEPTOS DEVOPS

CI: Integrar cambios a menudo + pruebas automáticas.

CD (Delivery): App lista para desplegar; producción requiere aprobación manual.

CD (Deployment): Despliegue a producción totalmente automático.

IaC: Infraestructura como Código (Terraform, CloudFormation).

Observabilidad: Métricas (Prometheus) + Logs (Loki/ELK) + Trazas (Jaeger).

Rollback: Volver a versión estable anterior de forma inmediata.

Artefacto: Producto del pipeline (imagen Docker, JAR, WAR, binario).

✅ BUENAS PRÁCTICAS

- ✓ Pipeline como código (Jenkinsfile en Git).
- ✓ Imagen inmutable: misma de staging a producción.
- ✓ No usar `:latest`; etiquetar con `BUILD_ID` o hash del commit.
- ✓ Credenciales en Jenkins (Credentials Binding), nunca en código.
- ✓ Idempotencia: Terraform y pipelines repetibles sin efectos laterales.
- ✓ Separación de entornos (redes, variables de entorno).
- ✓ Limpieza periódica: `docker system prune -f`.